Cuenta de GitHub y primer repositorio

Programación de Inteligencia Artificial · Unidad 1 · Apartado 1

Daniel Rubio Javier

1. Mi cuenta de GitHub

Para esta tarea necesitaba una cuenta de GitHub. Ya tenía una creada con la cuenta de Google que uso para el instituto y vinculada a ella, así que he utilizado esa misma cuenta. Mi nombre de usuario es dblondj.

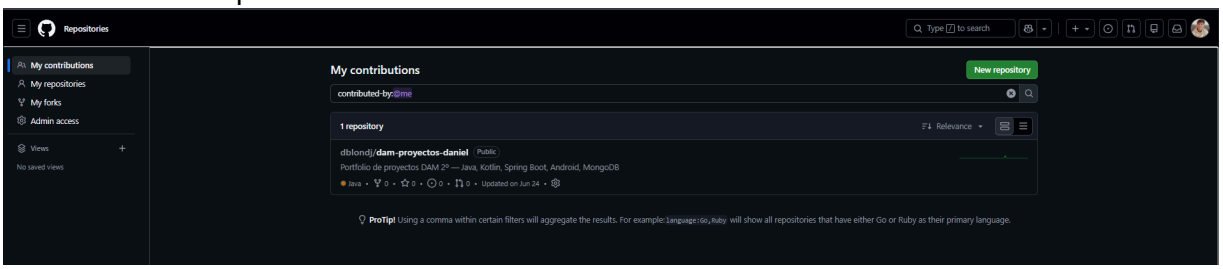
Así se crea una cuenta de este modo, que es como hice la mía:

1. Entré en **https://github.com/signup** desde el navegador.
2. Elegí **Continue with Google** para registrarme con mi cuenta de Google del instituto.
3. Seguí los pasos del formulario: comprobé el correo, elegí un nombre de usuario libre y resolví la verificación que pide GitHub para comprobar que no soy un robot.
4. Verifiqué la dirección de correo. Sin este paso no se pueden hacer algunas acciones básicas, como crear un repositorio.
5. Cuando GitHub preguntó si soy alumno o profesor, pulsé la opción **Skip personalization** de la parte inferior de la pantalla, como indica el enunciado.
6. Dejé el plan gratuito (Free) y entré en la página principal de GitHub con mi usuario: la cuenta quedó creada y enlazada con mi cuenta de Google.

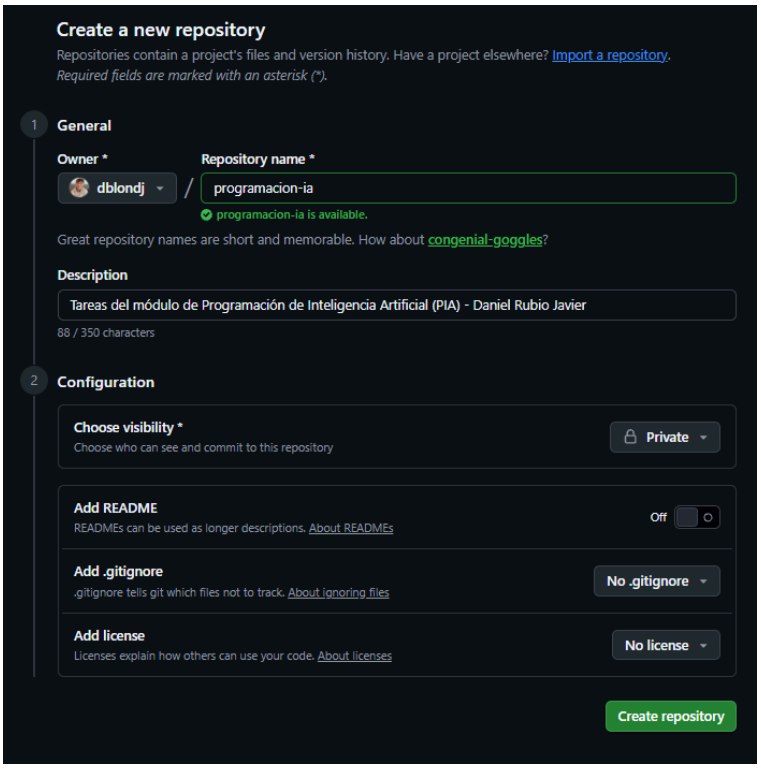
2. Mi primer repositorio

Ahora voy a crear el repositorio que utilizaré para el ejercicio.

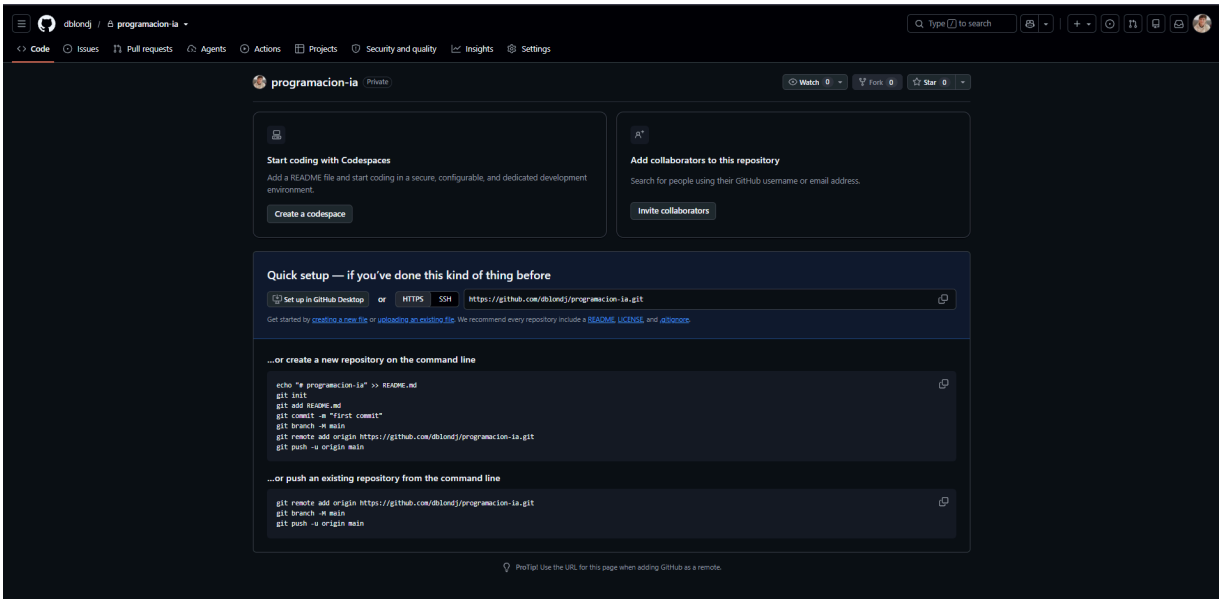
Entramos en la parte de todos los repositorios del menú lateral y vamos a hacer click en el botón de nuevo repositorio verde.



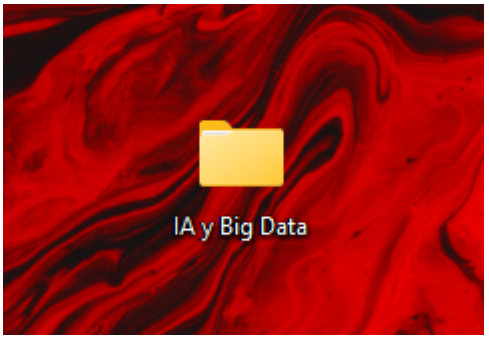
Rellenamos con los datos que queremos que tenga mi repositorio, y en mi caso lo voy a poner en privado porque solo yo quiero acceder a los datos que haya en el repositorio.

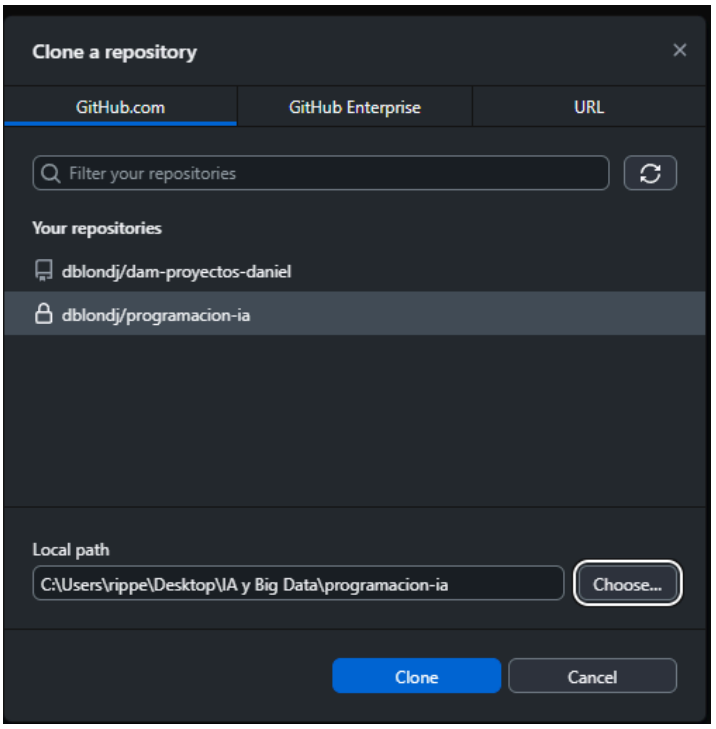


Tras pulsar "Create repository", GitHub creó el repositorio "programacion-ia" en mi cuenta (dblondj), con visibilidad privada. Al estar recién creado y vacío, GitHub muestra una pantalla de configuración rápida con la URL del repositorio, opciones para crear o subir archivos desde la web, las instrucciones para subirlo con comandos de Git y un botón para invitar colaboradores. Desde aquí ya se pueden añadir los archivos de la tarea.



Vale ya esta creado el repositorio, así que me voy a crear una carpeta en mi escritorio para que de esta manera las cosas que realicé en mi ordenador y suba a esa carpeta las pueda subir a Github, la aplicación que utilizo para que se “enlacen” es Github Desktop





3. Apartado 2: ejercicios de Python

Los tres ejercicios están guardados como archivos .py en la carpeta "unidad1" del repositorio. Para probarlos utilicé Visual Studio Code: abrí la carpeta "unidad1" (Archivo > Abrir carpeta), abrí cada archivo en el editor y lo ejecuté con el botón de "Run Python File" (el triángulo de arriba a la derecha). El resultado aparece en la terminal integrada de la parte inferior.

Problema 1: procesamiento de una lista de enteros

La función procesar_lista recorre la lista con un bucle for. Por cada número comprueba con un if que no sea negativo y que todavía no esté en la lista de resultado; si cumple las dos cosas, lo añade con append. Al terminar ordena el resultado de menor a mayor con sort(). Con el ejemplo del enunciado, [4, -1, 2, 4, 3, -5, 2] devuelve [2, 3, 4].

Codigo:

ejercicio1_ordenar_lista.py > ...

```python
# Problema 1: procesamiento de una lista de enteros

def procesar_lista(lista):
    resultado = []
    for numero in lista:
        # Guardo el número solo si no es negativo y si todavía no está en la lista
        if numero >= 0 and numero not in resultado:
            resultado.append(numero)
    # Ordeno los números de menor a mayor
    resultado.sort()
    return resultado


# Programa de prueba con el ejemplo del enunciado
ejemplo = [4, -1, 2, 4, 3, -5, 2]
print("Lista original:", ejemplo)
print("Lista procesada:", procesar_lista(ejemplo))
```

Ejecución:

```
$ C:/Users/rippe/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/rippe/Desktop/IA y Big Data/programacion-ia/unidad1/ejercicio1_ordenar_lista.py"
Lista original: [4, -1, 2, 4, 3, -5, 2]
Lista procesada: [2, 3, 4]
```

Problema 2: frecuencia de palabras en un texto

La función frecuencia_palabras crea un diccionario con cada palabra buscada (en minúsculas) y valor 0. Después abre el fichero y lo recorre línea a línea: pasa la línea a minúsculas, quita los signos de puntuación con replace y separa las palabras con split. Si una palabra del texto está en el diccionario, le suma 1. Al final el programa de prueba muestra las palabras ordenadas. La palabra "java" sale con 0 porque no aparece en el texto.

Codigo:

ejercicio2_frecuencia_palabras.py > ...

```python
# Problema 2: frecuencia de palabras en un texto

def frecuencia_palabras(palabras, ruta):
    # Creo el diccionario con cada palabra buscada y frecuencia 0
    frecuencias = {}
    for palabra in palabras:
        frecuencias[palabra.lower()] = 0

    # Signos de puntuación que hay que quitar del texto
    signos = ".,;:!?¡¿()\"'-"

    fichero = open(ruta, "r", encoding="utf-8")
    for linea in fichero:
        linea = linea.lower()  # todo a minúsculas
        for signo in signos:
            linea = linea.replace(signo, "")  # quito los signos de puntuación
        for palabra in linea.split():
            # Si la palabra está en el diccionario, sumo 1 a su frecuencia
            if palabra in frecuencias:
                frecuencias[palabra] = frecuencias[palabra] + 1
    fichero.close()

    return frecuencias


# Programa de prueba
buscadas = ["Python", "inteligencia", "la", "java", "artificial"]
resultado = frecuencia_palabras(buscadas, "texto_ejemplo.txt")

# Muestro las palabras ordenadas
for palabra in sorted(resultado):
    print(palabra + ": " + str(resultado[palabra]))
```

TXT:

texto_ejemplo.txt

```
La inteligencia artificial es una rama de la informática.
La programación de la inteligencia artificial usa Python.
Python es un lenguaje sencillo y la comunidad de Python es enorme.
```

Ejecución:

```
$ C:/Users/rippe/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/rippe/Desktop/IA y Big Data/programacion-ia/unidad1/ejercicio2_frecuencia_palabras.py"
artificial: 2
inteligencia: 2
java: 0
la: 5
python: 3
```

Problema 3: trabajo con conjuntos

Creé la función operaciones_conjuntos, que recibe dos listas de enteros, las convierte en conjuntos (set) y calcula con ellos la intersección (operador &, elementos comunes), la unión (operador |, todos los elementos sin duplicados) y la diferencia simétrica (operador ^, elementos que están en uno de los conjuntos pero no en ambos). La función devuelve un diccionario con los tres resultados. Para probarla utilicé las listas [1, 2, 3, 4, 4] y [3, 4, 5, 6], y obtuve: intersección [3, 4], unión [1, 2, 3, 4, 5, 6] y diferencia simétrica [1, 2, 5, 6].

Código:

ejercicio3_conjuntos.py > ...

```python
# Problema 3: trabajo con conjuntos

def operaciones_conjuntos(lista1, lista2):
    # Convierto las listas en conjuntos (set)
    conjunto1 = set(lista1)
    conjunto2 = set(lista2)

    resultado = {}
    # Elementos que están en los dos conjuntos
    resultado["interseccion"] = sorted(conjunto1.intersection(conjunto2))
    # Todos los elementos sin duplicados
    resultado["union"] = sorted(conjunto1.union(conjunto2))
    # Elementos que están en uno u otro, pero no en ambos
    resultado["diferencia_simetrica"] = sorted(conjunto1.symmetric_difference(conjunto2))
    return resultado


# Programa de prueba
lista_a = [1, 2, 3, 4, 4]
lista_b = [3, 4, 5, 6]
print("Lista 1:", lista_a)
print("Lista 2:", lista_b)
resultado = operaciones_conjuntos(lista_a, lista_b)
print("Intersección:", resultado["interseccion"])
print("Unión:", resultado["union"])
print("Diferencia simétrica:", resultado["diferencia_simetrica"])
```

Ejecución:

```
$ C:/Users/rippe/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/rippe/Desktop/IA y Big Data/programacion-ia/unidad1/ejercicio3_conjuntos.py"
Lista 1: [1, 2, 3, 4, 4]
Lista 2: [3, 4, 5, 6]
Intersección: [3, 4]
Unión: [1, 2, 3, 4, 5, 6]
Diferencia simétrica: [1, 2, 5, 6]
```

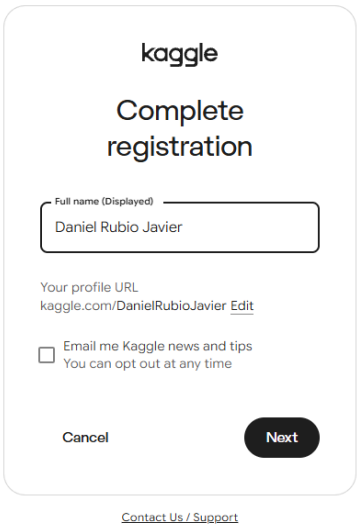
Los tres archivos los subí a la carpeta "unidad1" del repositorio.

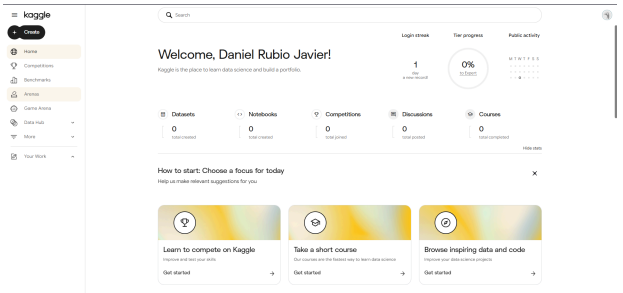


4.Apartado 3: Consultar competición en plataforma de IA Kaggle.

1.Crear la cuenta

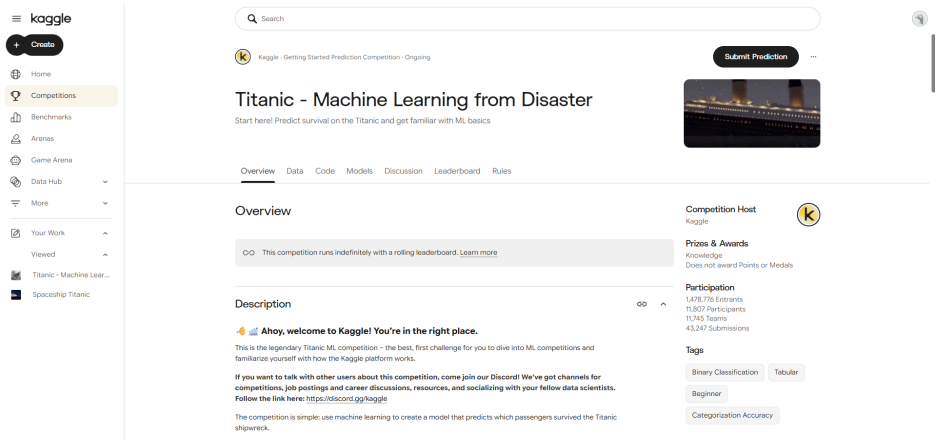
Entré en kaggle.com y pulsé Register para crear mi cuenta. Completé el registro con mi nombre (Daniel Rubio Javier), que se muestra en el perfil, y pulsé Next. Tras terminar, Kaggle me llevó a la página de inicio con el mensaje "Welcome, Daniel Rubio Javier!", lo que confirma que la cuenta quedó creada.





2.Acceder a una competición activa

En el menú lateral entré en Competitions y elegí "Titanic - Machine Learning from Disaster", una competición de tipo Getting Started. En la parte superior aparece como "Ongoing" y en la descripción indica que se ejecuta de forma indefinida, así que está activa. El objetivo es crear un modelo de aprendizaje automático que prediga qué pasajeros sobrevivieron al hundimiento del Titanic.



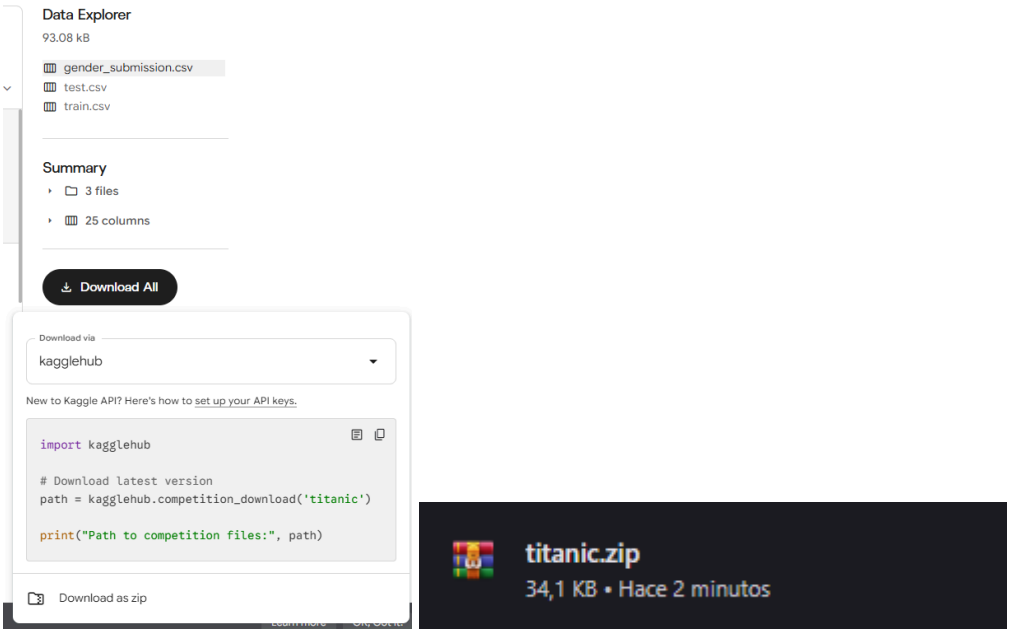
3. Descargar el dataset

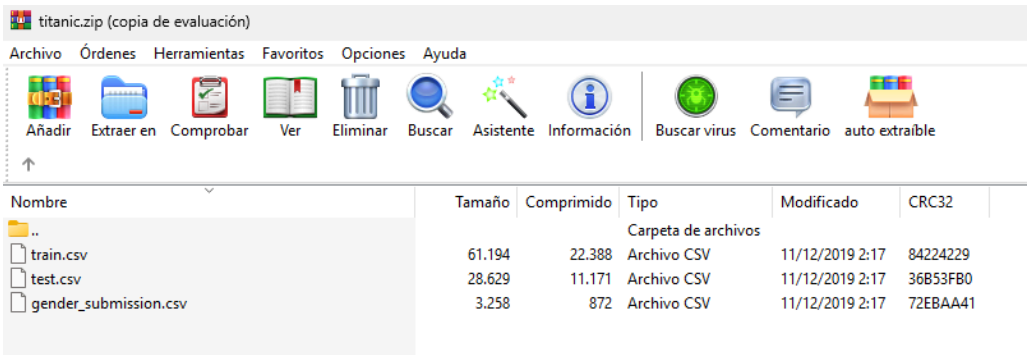
Dentro de la competición entré en la pestaña Data. No me pidió unirme a la competición, porque el botón de la parte superior ya mostraba "Submit Prediction". Bajé hasta el final de la página y pulsé Download All, y se descargó el archivo titanic.zip (34,1 KB). Al abrirlo vi que contiene tres archivos:

- train.csv: datos de entrenamiento, con la columna de supervivencia de cada pasajero.

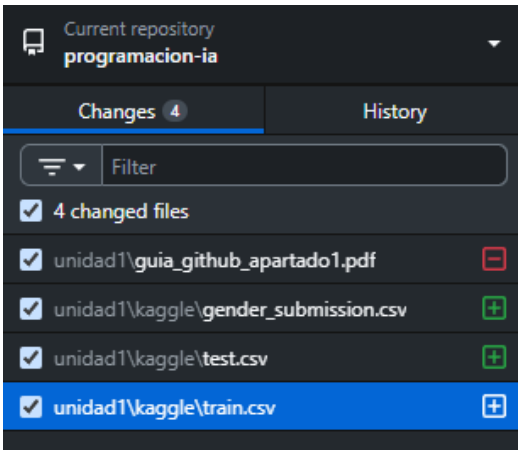
- test.csv: datos de prueba, sin la columna de supervivencia, que es la que hay que predecir.

- gender_submission.csv: ejemplo del formato del archivo que hay que enviar a la competición.





Por último, descomprimí los archivos y los subí al repositorio con GitHub Desktop, dentro de la carpeta "unidad1/kaggle". En la segunda captura se ve que los tres archivos ya están en GitHub.


Current repository
programacion-ia
Changes 4
History
Filter
4 changed files
unidad1\guia_github_apartado1.pdf
unidad1\kaggle\gender_submission.csv
unidad1\kaggle\test.csv
unidad1\kaggle\train.csv


programacion-ia / unidad1 / kaggle /
Add file
dblondj subida de los archivos del dataset
f92e26c · now
History
Name
Last commit message
Last commit date
..
gender_submission.csv
subida de los archivos del dataset
now
test.csv
subida de los archivos del dataset
now
train.csv
subida de los archivos del dataset
now